DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course: CSE 489
Name: Imtiaz Hossain
ID: 23101137
Email: imtiaz.hossain@g.bracu.ac.bd

Project Title: *Portico, a real estate investment portfolio platform for Android*

1. Introduction

Portico is a real estate investment portfolio platform for Android. It lets an investor record properties across several countries and currencies, and it computes the returns, cashflow and tax that those records imply. Every figure the application shows is derived from the member's own records rather than typed in by hand, and the arithmetic behind each figure can be opened and inspected rather than taken on trust.

The application targets Android 8.0, API level 26, and above. It is written in Kotlin with Jetpack Compose and Material 3, and it is backed by a serverless bridge that performs every operation the client must not be trusted to perform: authentication checks, plan limits, role changes, private file access and account deletion.

This report covers the implemented features, the architecture and data model, the key implementation details, how the work was tested, and the problems encountered while building it.

2. Implemented Features

The following features are implemented and working end to end.

Area	What works
Registration and login	Clerk email and password, Google OAuth, email code verification, in-app password change, password reset, live session management and full account deletion.
Portfolio management	Six-step property capture with live analysis, photographs, search, filter, sort, edit and delete, with the Free-plan limit enforced on the server.
Financial analysis	ROI, capital ROI, cash-on-cash, cap rate, gross and net yield, monthly and annual cashflow, value history and a gross-to-net waterfall.
Tax computation	Nine jurisdictions across Bangladesh, Uruguay and Argentina, with editable assumptions and the arithmetic shown behind every rate.
Reports	Performance, cashflow, allocation, property comparison and gross-versus-net analysis, normalized across three currencies.
Documents	Private library backed by Vercel Blob, upload, download and delete through the Android system picker, categories and an in-app PDF reader.
Portico Intelligence	On-device portfolio analysis that shows its working, with optional cloud analysis through the Gemma API that sends computed figures only.
Plans and payments	Free and Pro rules, receipts, billing history, cancellation and resume. Checkout runs either a built-in sandbox or SSLCommerz's hosted gateway, chosen by the server. Verified end to end against SSLCommerz's sandbox with a live bKash transaction. No card is charged on either route.
Organisations	Owner, admin, analyst and viewer roles enforced on the server, with invitations by email through Clerk lookup.
Platform administration	Cross-tenant console for an allowlisted operator: member list from Clerk, counts from Firestore collection groups, plan changes between Free and Pro, and sign-in locking applied through Clerk. Granted by a deployment variable and never from inside the application.
Portability	CSV export and import with duplicate detection and line-level rejection reporting.
Localization	English and Bangla across 770 interface strings, switchable in-app and from Android per-app language settings.
Notifications	Firebase Cloud Messaging with owner-bound Installation ID registration.

3. System Architecture

The application is organised into three Android layers and one server component. The separation is not decorative: it is what makes the financial arithmetic testable without an emulator.

3.1 Domain layer

The domain package is pure Kotlin and imports nothing from Android. It holds Finance.kt for returns and cashflow, Tax.kt for the jurisdiction rules, Analyst.kt for the on-device assistant, Exchange.kt for currency normalization, Payments.kt for the sandbox processor, and Models.kt for the record types. Because it has no Android dependency, every tax rule and every return calculation runs on the JVM in seconds during a normal unit test.

3.2 Data layer

PorticoStore holds an account-scoped DataStore cache so the application works offline. FirestoreWorkspaceRepository mirrors that state into normalized Firestore collections. PorticoBackend and PorticoFiles talk to the serverless bridge. PorticoExport and PorticoImport handle CSV portability.

3.3 Presentation layer

Sixteen Compose screens sit on a shared design system in ui/design: Foundation.kt for typography and figure rendering, Charts.kt for the visualisations, States.kt for empty, loading and error states, and PdfView.kt for the in-app document reader. Screen state is held in AppState.kt and passed down rather than read from globals.

3.4 Serverless bridge

Ten TypeScript functions run on Vercel: firebase-token, properties, subscription, files, notifications, workspace, account, assistant, organizations and market. Each verifies the caller's Clerk session token before doing anything. The bridge exists because a client cannot be trusted to enforce its own limits: an attacker can modify an APK, but cannot modify the server.

4. Database Schema

Data is stored in Cloud Firestore under two top-level trees, plus a public tree for reference data. The full schema diagram is included in Doc/diagrams as Schema_diagram_portico_Imtiaz Hossain.pdf.

```
users/{userId}/
  workspace/{snapshot}      migrated schema-v1 snapshot
  properties/{propertyId}   one document per property
  payments/{paymentId}      sandbox receipts and billing history
  devices/{deviceId}        FCM Installation IDs, owner bound
  settings/{document}       quota, subscription, rateLimit

organizations/{orgId}/
  members/{userId}          role: OWNER | ADMIN | ANALYST | VIEWER

public/market/{document}    reference data, observed or modelled
```

The security rules are the important half of the schema. A member may read and write their own property and payment records, but the settings documents that carry quota and subscription state are readable by their owner and writable by nobody. Every organisation document is readable by its members and writable by nobody. All writes to those paths are routed through the bridge, which applies the rules before writing.

```
// firestore.rules
match /settings/{document} {
  allow read:  if owns(userId);
  allow write: if false;          // server-owned, never client-writable
}
```

5. Wireframe

The wireframe was produced before implementation and is included in Doc/diagrams as Wireframe_diagram_portico_Imtiaz Hossain.pdf. It covers the onboarding flow, the dashboard, the portfolio list, the property detail tabs, the six-step property capture, the reports surfaces and the account screens.

6. Application Screenshots

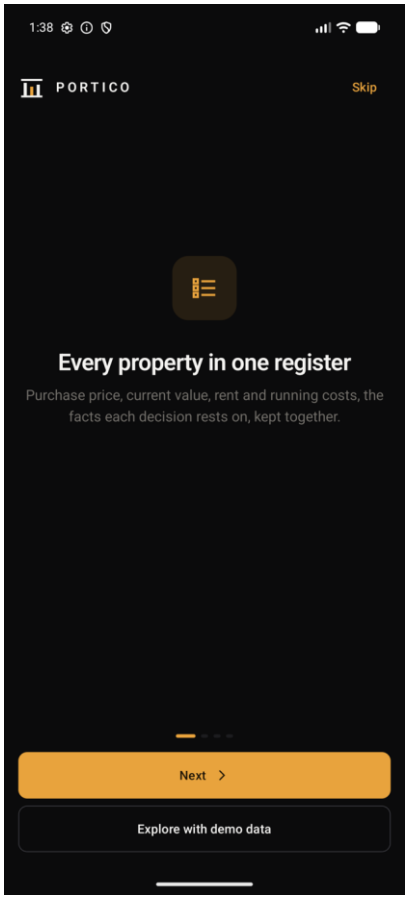

Figure 1. Onboarding

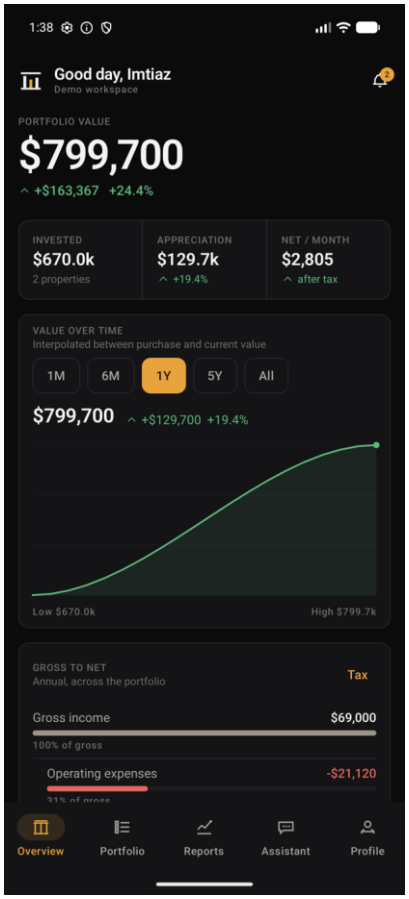

Figure 2. Dashboard, roll-up across three currencies

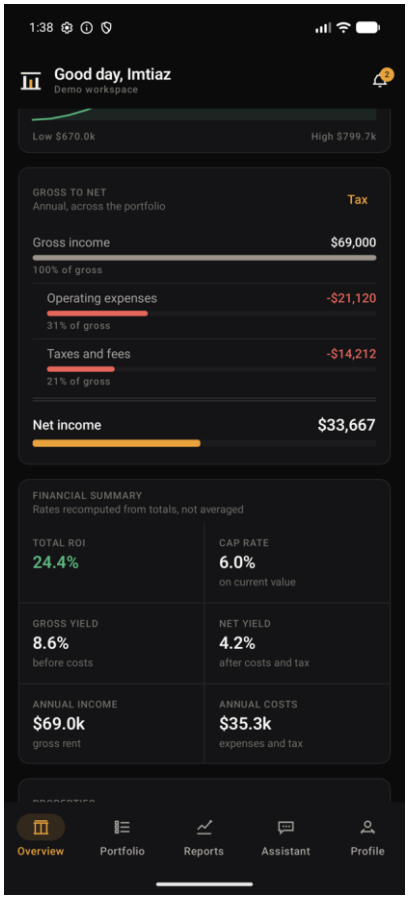

Figure 3. Gross-to-net waterfall, every deduction itemised

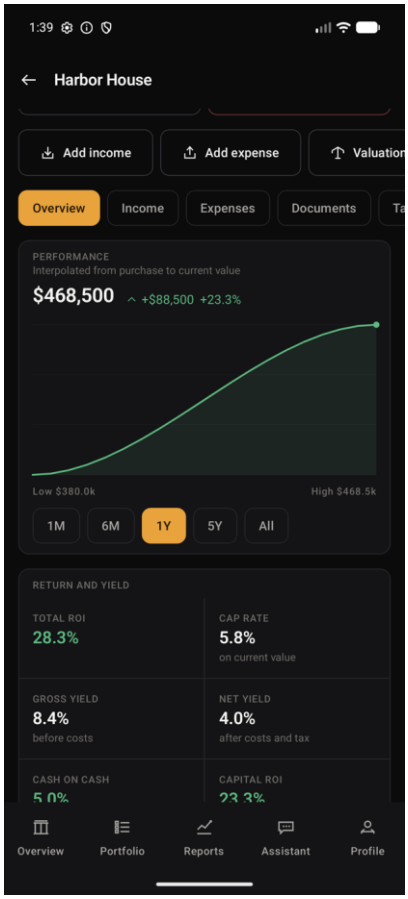

Figure 4. Property detail

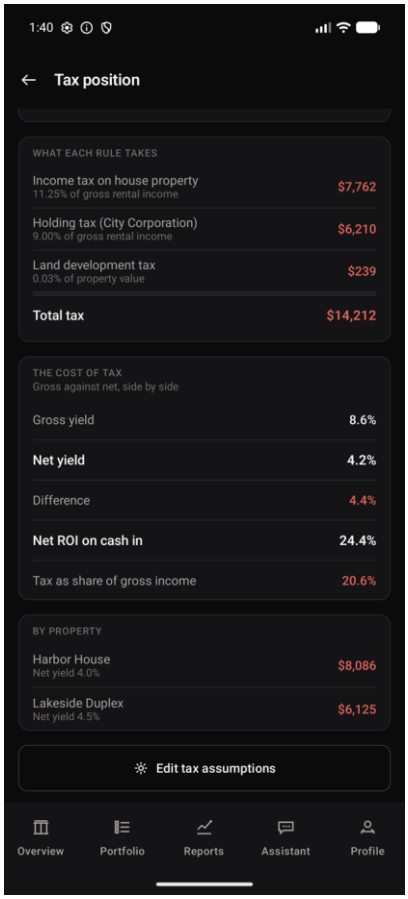

Figure 5. Bangladesh tax rules, arithmetic shown

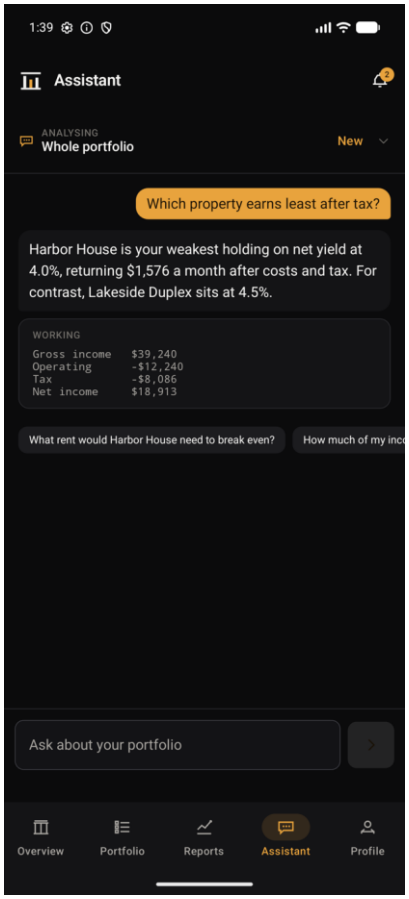

Figure 6. Portico Intelligence, working expanded

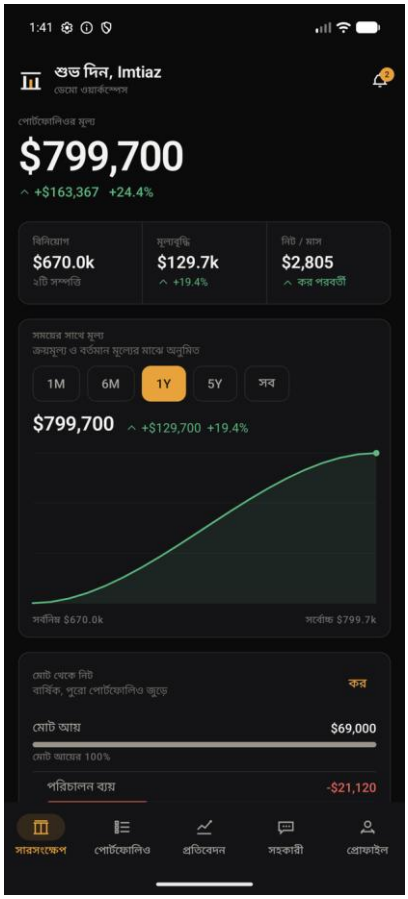

Figure 7. Bangla interface, figures held in Latin digits

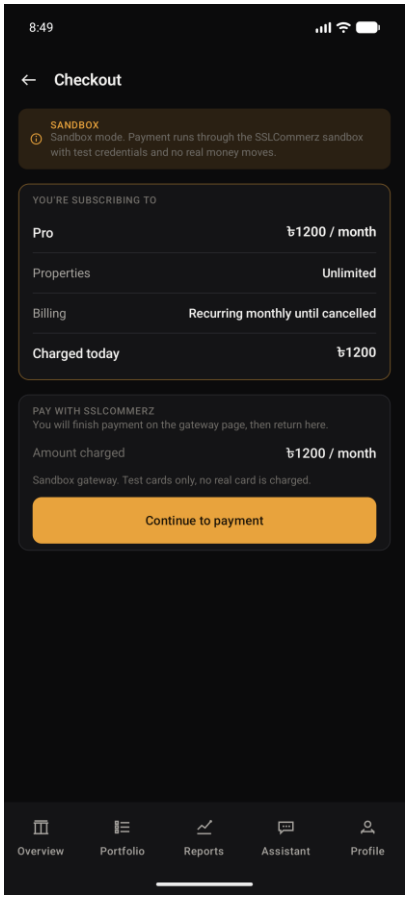

Figure 8. Gateway checkout, priced in the currency it settles in

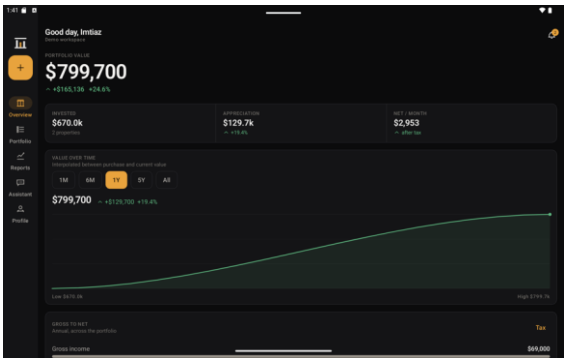

Figure 9. Tablet dashboard

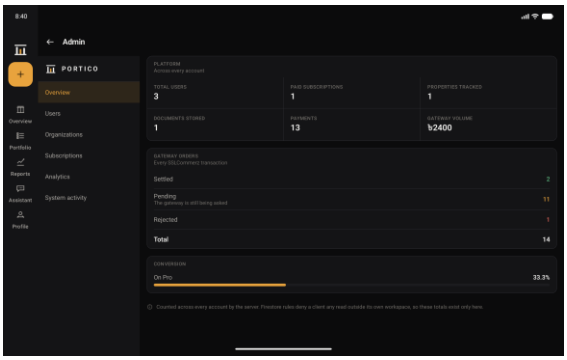

Figure 10. Platform console, counted across every account

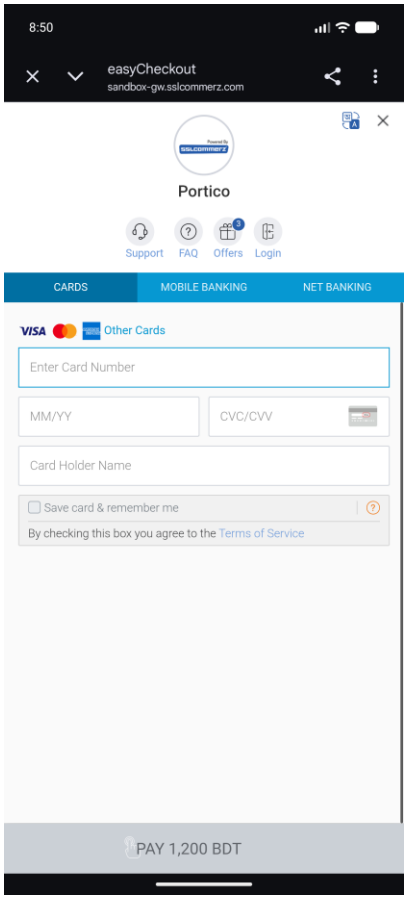

Figure 11. SSLCommerz hosted page, sandbox host

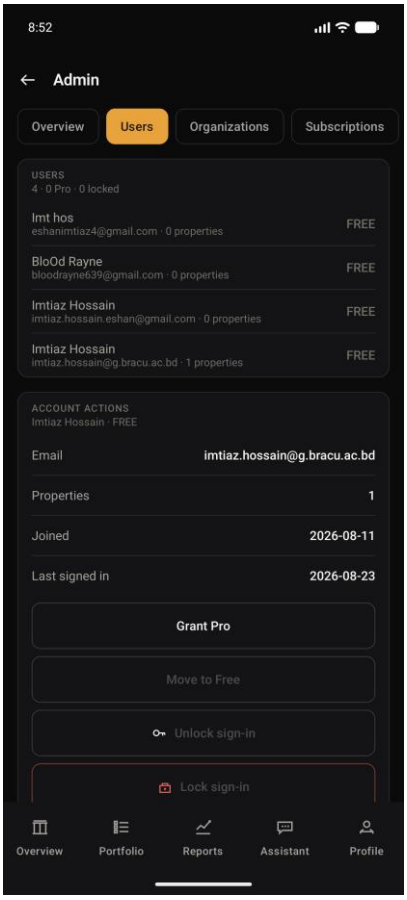

Figure 12. Administrator actions on a member

7. Key Implementation Details

7.1 Verifying the caller on the server

A login screen proves nothing on its own, because the application is the thing doing the asking. Every call to the bridge carries the Clerk session token, and the server verifies it against Clerk's public keys and reads the user id out of the signed token. It never reads a user id from the request body, because that is the one field a modified client could falsify.

```
// bridge/lib/clerk-auth.ts
export async function verifyClerkRequest(request: VercelRequest): Promise<string> {
  const token = bearerToken(request.headers.authorization);
  if (!token) throw new ClerkAuthorizationError(missing_session_token);

  const issuer = clerkIssuer();
  const {payload} = await jwtVerify(token, clerkJwks(issuer), {
    issuer,
    algorithms: [RS256],
  });
```

```
  const userId = payload.sub;
  if (!userId || !/^[A-Za-z0-9_-]+$/.test(userId)) {
    throw new ClerkAuthorizationError(invalid_user_identity);
  }
  return userId;
}
```

7.2 Enforcing the plan limit in a transaction

The Free plan allows two properties. Checking this on the client would be cosmetic, and checking it on the server without a transaction would still allow two devices to add a property at the same instant and both pass. The check therefore runs inside a Firestore transaction.

```
// bridge/api/properties.ts
const FREE_PROPERTY_LIMIT = 2;

await db.runTransaction(async (transaction) => {
  if (!isPro && currentCount + records.length > FREE_PROPERTY_LIMIT) {
    throw new ApiError(409, plan_limit_reached,
      Upgrade to Pro to add another property);
  }
  // ... write the records
});
```

7.3 Role ranking

Organisation roles are ranked, so privilege escalation is a comparison rather than a list of special cases. Nobody may grant a role at or above their own, nobody may act on a member who outranks them, and the last owner cannot be removed.

```
// bridge/api/organizations.ts
const RANK: Record<string, number> =
  {OWNER: 4, ADMIN: 3, ANALYST: 2, VIEWER: 1};

if (rankOf(role) >= rankOf(membership.role)) {
  throw new ApiError(403, 'role_exceeds_your_own');
}
if (rankOf(target.role) >= rankOf(membership.role)) {
  throw new ApiError(403, 'target_outranks_you');
}
```

7.4 Per-app language

Switching language required calling the platform LocaleManager directly on API 33 and above. The reason is explained in section 9.

```
// app/src/main/java/com/portico/android/ui/Language.kt
fun apply(context: Context, language: AppLanguage) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        val manager = context.getSystemService(LocaleManager::class.java) ?: return
        manager.applicationLocales =
```

```
            if (language.tag.isEmpty()) LocaleList.getEmptyLocaleList()
            else LocaleList.forLanguageTags(language.tag)
    } else {
        AppCompatDelegate.setApplicationLocales(...)
    }
}
```

7.5 Keeping figures in Latin digits

Formatting against the device locale renders numbers in Bengali numerals under a Bangla locale. A portfolio figure is not the place to surprise somebody, so all figures format against Locale.ROOT.

```
// app/src/main/java/com/portico/android/domain/Finance.kt
// Figures are formatted against Locale.ROOT, never the device locale.
private val FIGURES: java.util.Locale = java.util.Locale.ROOT
```

7.6 Granting platform administration

Every other endpoint answers only for the caller's own records. The platform console deliberately reads across all of them, which makes the privilege check the whole security story rather than a detail of it. Administration is therefore granted by a deployment variable and nothing else: it cannot be requested through an endpoint, set from inside the application, or written to Firestore by any client, because a privilege that the governed thing can grant itself is not a privilege.

```
// bridge/lib/platform-admin.ts
export function requirePlatformAdmin(userId: string): void {
  const allowed = adminUserIds();
  if (allowed.length === 0) {
    throw new ApiError(503, 'admin_not_configured');
  }
  if (!allowed.includes(userId)) {
    // Deliberately the same shape as any other refusal.
    // Confirming an admin surface exists is itself information.
    throw new ApiError(403, 'not_a_platform_administrator');
  }
}
```

An administrator can move a member between Free and Pro and can lock or unlock sign-in, which Clerk applies. Deleting somebody else's account is deliberately absent: locking is reversible and leaves their records intact, which is why it is the strongest action offered. The counts come from Firestore collection-group aggregates and the member list from Clerk, because Clerk owns identity while Firestore owns the portfolio, and neither is asked a question the other one answers.

8. Testing

One hundred and nine unit tests and seven instrumented tests all pass. The unit tests run on the JVM because the domain layer has no Android dependency; the instrumented tests drive the real application on a device.

Test class	What it covers
FinanceTest	ROI, cap rate, yields, cashflow and the waterfall
TaxTest	All nine jurisdictions, verified against hand-computed figures
ExchangeTest	Cross-currency normalization and rate invariance
AccessControlTest	Role ranking, privilege escalation, last-owner protection
AnalystTest	On-device assistant answers and the working it shows
SandboxPaymentTest	Checkout outcomes, receipts, cancellation and resume
PorticoImportTest	CSV import, duplicate detection, line-level rejection
AssistantPrivacyTest	Asserts addresses and notes never enter the cloud payload
PorticoJourneyTest	Seven instrumented end-to-end journeys on a device

Continuous integration runs three jobs on every push: the Android build with unit tests, a TypeScript type check on the bridge, and a scan that fails the build if a credential ever lands in the repository.

One result is worth recording honestly. Running the signed release build on a physical device found three defects that the whole unit suite had missed, including both localization defects described below. Automated tests are necessary and they are not sufficient.

9. Challenges Faced

9.1 A language switch that silently did nothing

AppCompatDelegate.setApplicationLocales is the documented way to change the in-app language. It does nothing at all when the host activity is a ComponentActivity rather than an AppCompatActivity, which is what a Compose-only application has. There is no crash, no warning and no log line; the call simply returns. The fix was to call the platform LocaleManager directly on API 33 and above and keep the AppCompat backport only for older devices.

9.2 A currency figure that broke in half

Under a Bangla locale the dashboard headline rendered as a broken figure with the taka sign stranded on its own line. My first diagnosis was wrong: I assumed a sizing problem and made it worse by introducing silent truncation. The real cause is that Android treats the boundary between the taka sign and the following digit as a mandatory line-break opportunity, because they belong to different scripts. FigureText now measures the symbol and the digits separately and lays them out as one unbreakable row.

9.3 Localizing without breaking the architecture

Around 230 strings in the domain and data layers remain in English. They cannot use Android string resources, because those layers deliberately have no Android dependency, and that separation is what makes the finance engine testable on the JVM. Resolving it means either giving up the separation or returning keys for the interface to resolve. This is a design decision rather than a mechanical task, and the README records it plainly rather than claiming complete coverage.

9.4 A dependency conflict between module systems

The account deletion endpoint failed in production with ERR_REQUIRE_ESM. The cause was that jwks-rsa declares a dependency on jose version 6, which is ESM-only, while requiring it through CommonJS. The resolution was an npm overrides block pinning jose to 4.15.9 across the whole tree, followed by a clean reinstall.

9.5 Instrumented tests against a new API level

All seven instrumented tests failed with NoSuchMethodException on InputManager.getInstance. Espresso 3.6.1 does not support API level 37. Upgrading to Espresso 3.7.0 and androidx.test 1.7.0 resolved it. A second failure followed, where the tests inherited whatever locale the application had last been left in; the fix was to pin the locale inside the test rather than depend on external state.

9.6 Release build correctness under R8

Enabling R8 minification and resource shrinking risks breaking kotlinx-serialization, which relies on generated serializers. Rather than assume the keep rules were correct, I verified empirically: I set the display currency to BDT in the minified release build, force-stopped the application, and relaunched it. The figure returned correctly, which proves both serialization and deserialization survive minification.

9.7 Credentials in version control

The Firebase configuration file was committed early in development and remains reachable in the public repository history. An Android API key is designed to ship inside every APK and is not a secret, so the correct remedy is restriction rather than rotation: the key is now restricted to the application's package name and signing certificate, and to the four APIs it actually needs. The file is now gitignored and CI fails the build if a credential reappears.

9.8 A payment flow that reported success and settled nothing

Integrating SSLCommerz was the hardest problem in the project, and almost none of the difficulty was the parts the documentation covers. Opening a session and validating a payment worked early. What did not work was everything around them.

The gateway's own callback never arrived. The design assumed it would: the payment is settled when SSLCommerz posts to the server. Their sandbox simply does not send it, so a member who had genuinely paid sat watching a spinner forever. The fix was to stop treating settlement as something that happens to us and start asking for it, by querying SSLCommerz directly by transaction id when the member returns. Both routes end in the same function, so what may be credited, and on what evidence, does not depend on which one arrived.

Asking once was not enough either. The gateway is not ready to answer for a transaction it has only just redirected away from, so the first query reliably returned nothing, and treating that as the final answer failed a successful payment. Checkout now asks repeatedly across a short poll.

The defect that cost the most time was mine and was four words long. The code that handles the return from the gateway consumed the value it was keyed on, which made the interface framework cancel the very routine doing the work and restart it against nothing. Everything after the first suspension point was silently discarded. Every symptom pointed at the payment gateway, and the cause was a lifecycle mistake in my own screen.

The lesson worth keeping is that an integration is not finished when the happy path returns a success code. It is finished when the failure modes have been seen: the callback that never comes, the query that is too early, the redirect that proves nothing.

10. Online Resources Used

a) Reference

- Android Developers documentation and Jetpack Compose guides: developer.android.com
- Material Design 3 guidelines: m3.material.io
- Clerk Android SDK documentation: clerk.com/docs
- Firebase documentation for Authentication, Firestore, Cloud Messaging and App Check: firebase.google.com/docs
- Vercel Functions and Vercel Blob documentation: vercel.com/docs
- Google AI for Developers, Gemma model reference: ai.google.dev

b) Github links

- Project repository: github.com/ImtiazHossain-Eshan/Portico_Android
- Firebase Android SDK: github.com/firebase/firebase-android-sdk
- Coil image loading for Compose: github.com/coil-kt/coil
- Vercel Functions runtime: github.com/vercel/vercel

11. Future Enhancements

The following would improve the current system.

- Complete Bangla coverage for the domain and data layers, once the design question in section 9.3 is settled.
- Replace the modelled market reference data with a live property data feed. The server already declares whether each figure is observed or modelled, so the contract does not change.
- Share the finance engine with an iOS client through Kotlin Multiplatform, which is what the Android-free domain layer was built to allow.

12. Conclusion

Portico implements registration and login through Clerk, a complete property portfolio with real financial and tax analysis, private document storage, an on-device assistant, organisations with server-enforced roles, and a bilingual interface. The design principle running through it is that the client is never the authority: the server verifies every caller, enforces every limit inside a transaction, and refuses writes to the records that matter.

The work that remains is documented honestly rather than hidden. Payment processing is sandboxed and labelled as such, market reference data is modelled and declares itself modelled, and the parts of the interface still in English are recorded in the README.